Preface

It is a capital mistake to theorize before one has data.
Sherlock Holmes

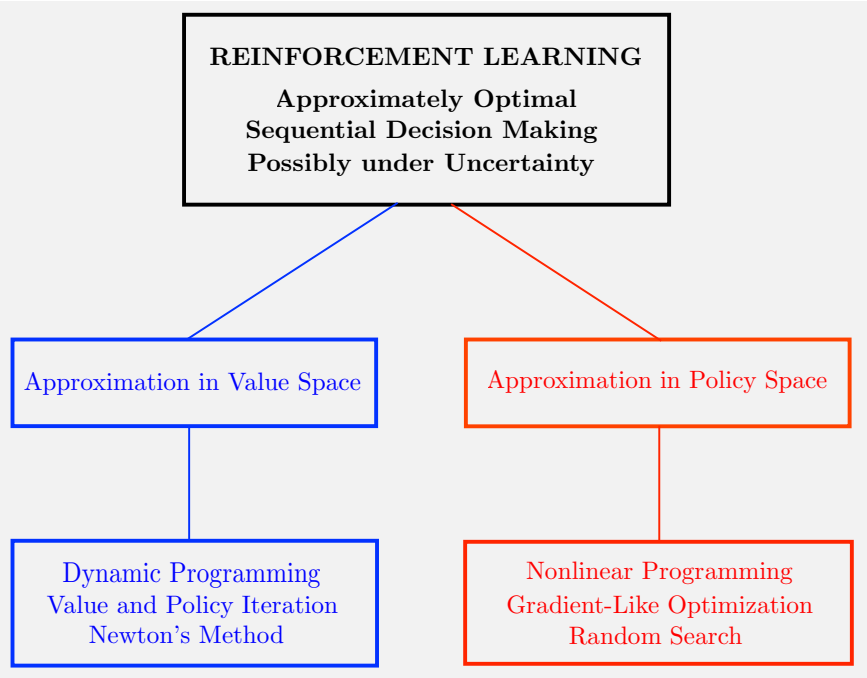
We are surrounded by data, but starved for insights.†

This book is based on lecture notes for the ASU research-oriented course on Reinforcement Learning (RL) that I have offered in each of the years from 2019 to 2025, as the field was rapidly evolving. The purpose of the book is to give an overview of the RL methodology, with a particular focus on problems of optimal and suboptimal control, as well as discrete optimization. More broadly, the book aims to provide a framework for structured thinking about RL and its connections to decision and control, which is grounded in mathematics, but is not dominated by it.

Generally, RL can be viewed as the art and science of approximately optimal sequential decision making for large and difficult problems, often in the presence of imprecisely known and changing environment conditions.‡ Dynamic Programming (DP), a well-established optimization methodology,

† Attributed to Jay Baer.

‡ The name "reinforcement learning" evolved from the concept of "reinforcement" in behavioral psychology. It refers to learning through interaction with the environment, shaped by rewards and penalties. The meaning of the name



deals with the same problem as RL, and forms its theoretical foundation. This is unlikely to change in the future, despite the current rapid pace of innovation. In fact, there are strong connections between sequential decision making and the new wave of technological progress, generative artificial intelligence, transformers, GPT applications, and natural language processing, as we will aim to show in this book.

Approximations in Value and Policy Space

In DP there are two principal objects to compute: the *optimal value function*, which provides the optimal cost that can be attained starting from any given initial state, and the *optimal policy*, which provides the optimal decision to apply at any given state and time. Unfortunately, the exact application of DP runs into formidable computational challenges, commonly known as the *curse of dimensionality*. To address these, RL aims to approximate the optimal value function and policy, by using manageable off-line and/or on-line computation, which often involves neural networks.

Thus there are two major methodological approaches in RL: *approximation in value space*, where we approximate in some way the optimal value function, and *approximation in policy space*, where we construct a

has significantly evolved since its initial popularization in the 1980s.

suboptimal policy by optimizing over a restricted set of policies; see the figure.† These two approaches can sometimes be combined to exploit the strengths of both. Generally, approximation in value space aligns more closely to the central DP ideas of value and policy iteration, which in turn connect closely to methods for solution of Bellman's equation using Newton's method.

On the other hand, approximation in policy space often relies on nonlinear programming ideas, and particularly gradient-like descent. In this connection it is worth noting two contexts:

(a) *Approximation of a target policy*, whereby we want to approximate a given policy. The policy is assumed to be accessible for sampling, and it can be used to obtain sample pairs, each consisting of a state and the control generated by the policy at that state. Many such sample pairs are collected and they are used to train a policy approximation with some form of supervised learning algorithm. Methods of this type are described in Section 3.4, and to some extent, in Sections 3.1-3.3 and 3.6.

(b) *Direct optimization over a parametric class of policies*. Here there is no target policy to approximate. Instead, we aim to approximate an optimal policy directly, through optimization of the cost function over a class of policies that depend on a parameter. This optimization is done with some kind of gradient-like algorithm (sometimes called a *policy gradient method*) or some form of random search. Methods of this type are described in Section 3.5.

From our point of view, RL consists of *all DP-like methods that aim to solve suboptimally problems formulated in terms of sequential decisions, as long as they involve approximation, either in value space or in policy space (or in both)*.‡ This definition is very broad and applies among others, to deterministic, stochastic, and minimax problem contexts not ordinarily viewed in terms of sequential decisions, such as integer programming and tuning of LLMs (see Chapter 2).

Let us finally note that while the connections between approximation in value and policy space are a recurring theme throughout our narrative, our presentation focuses primarily on approximation in value space. How-

† The terms "approximation in value space" and "approximation in policy space" were introduced by Bellman [Bel57], and are consistent with the name "neuro-dynamic programming" of the Bertsekas and Tsitsiklis book [BeT96] (the marriage of dynamic programming and the neural network technology).

‡ The name "reinforcement learning" describes well approximation in policy space and its gradient-like methodology: reinforcing/improving parametrically defined policies can be effected by changing parameters along their gradients. It describes less well approximation in value space, because reinforcing good value functions does not make much sense from a methodological point of view.

ever, the book is structured to allow instructors to go into approximation in policy space in greater depth, using any of a number of available sources.

Approximation in Value Space - Off-Line Training and On-Line Play

An important part of our development of the approximation in value space approach is a new conceptual framework, which aims to bridge the gaps between the artificial intelligence, control theory, and operations research views of our subject. This framework, the focus of the author's recent monograph "Lessons from AlphaZero for Optimal, Model-Predictive, and Adaptive Control," [Ber22a], centers on approximate forms of DP that are inspired by some of the major successes of RL involving games. Primary examples are the recent (2017) AlphaZero program (which plays chess), and the similarly structured and earlier (1990s) TD-Gammon program (which plays backgammon).

Our framework is couched on two general algorithms that are designed largely independently of each other and operate in synergy through the powerful mechanism of Newton's method, applied to solution of the fundamental Bellman equation of DP. We call these the *off-line training* and the *on-line play* algorithms. In the AlphaZero and TD-Gammon game contexts, the off-line training algorithm is the method used to teach the program how to evaluate positions and to generate good moves at any given position, while the on-line play algorithm is the method used to play in real time against human or computer opponents.

Our synergistic view of off-line training and on-line play is motivated by some striking empirical observations. In particular, both AlphaZero and TD-Gammon were trained off-line extensively using neural networks and an approximate version of the fundamental DP algorithm of policy iteration. Yet the AlphaZero player that was obtained off-line is not used directly during on-line play (it is too inaccurate due to approximation errors that are inherent in off-line neural network training). Instead, a separate on-line player is used to select moves, based on multistep lookahead minimization and a terminal position evaluator that was trained using a deep neural network, simultaneously with the off-line player. The on-line player performs a form of policy improvement, which is not degraded by neural network approximations. As a result, it greatly improves the performance of the off-line player.

Similarly, TD-Gammon performs on-line a policy improvement step using one-step or two-step lookahead minimization, which is not degraded by neural network approximations. To this end, it uses an off-line neural network-trained terminal position evaluator, and importantly it also extends its on-line lookahead by rollout (simulation with the one-step lookahead player that is based on the position evaluator). Thus in summary:

(a) The on-line player of AlphaZero plays much better than its extensively trained off-line player. This is due to the beneficial effect of exact policy improvement with long lookahead minimization, which corrects for the inevitable imperfections of the neural network-trained off-line player, and position evaluator/terminal cost approximation.

(b) The TD-Gammon player that uses long rollout plays much better than TD-Gammon without rollout. This is due to the beneficial effect of the rollout, which serves as a supplement and substitute for long lookahead minimization.

An important lesson from AlphaZero and TD-Gammon is that the performance of an off-line trained policy can be greatly improved by on-line approximation in value space, with long lookahead (involving minimization or rollout with the off-line policy, or both), and terminal cost approximation that is obtained off-line. This performance enhancement is often dramatic and is due to a simple fact, which is couched on algorithmic mathematics and is a focal point of our course: *approximation in value space with one-step lookahead minimization amounts to a step of Newton's method for solving Bellman's equation, while the starting point for the Newton step is based on the results of off-line training, and may be enhanced by longer lookahead minimization and on-line rollout*. Indeed the major determinant of the quality of the on-line policy is the Newton step that is performed on-line, while off-line training plays a secondary role by comparison.

An additional benefit of policy improvement by approximation in value space, not observed in the context of games (which have stable rules and environment), is that it works well with changing problem parameters and on-line replanning, similar to the methodology of indirect adaptive control. In particular, the Bellman equation is perturbed due to the parameter changes, but approximation in value space still operates as a Newton step. An essential requirement here is that a system model is estimated on-line through some identification method, and is used during the one-step or multistep lookahead minimization process. This may be a mathematical model represented by equations and probability distributions, or a computer model that involves simulation and/or neural network software.

In this book, we will describe the basic RL methodologies, and we will aim to explain (often with visualization) their effectiveness (or lack thereof) through the lens of the off-line and on-line synergy. In the process, we will bring out the strong connections between the artificial intelligence view of RL, the control theory view of sequential decision making, and the operations research view of discrete optimization. Moreover, we will describe a broad variety of algorithms that can be used for on-line play.

In particular, we will aim to show that the methodology of approximation in value space and rollout applies very broadly to deterministic and stochastic optimal control problems, involving both discrete and continuous search spaces, as well as finite and infinite horizon. We will also show that

our conceptual framework can be effectively integrated with other important methodologies such as multiagent systems and decentralized control, discrete and Bayesian optimization, two-person antagonistic games, and heuristic algorithms for discrete optimization.

Reinforcement Learning and Model Predictive Control

Significantly, the synergy between off-line training and on-line play also underlies Model Predictive Control (MPC), a major control system design methodology that has been extensively developed since the 1980s.† One of the major themes of this book is that the RL approximation in value space approach, as practiced by the artificial intelligence community, and MPC, as practiced by the decision and control community, *can be viewed as essentially identical methodologies at a mathematical and conceptual level*.‡

This is particularly true in the case where the system model (either mathematical or simulated or trained neural network) is known and not changing over time. Problems with an unknown, partially known, or time-varying model belong to the realm of adaptive control; see Section 1.6.8. Important methods here include PID control, direct adaptive control, and indirect adaptive control. The latter method (also known as "dual control" or "combined identification and control") is essentially MPC, augmented with an online system identification algorithm that continuously updates the system model using sensor measurements or other information. Note that approximation in policy space methods are ill-suited for adaptive control, since online retraining of policies is prohibitively sample-intensive.

The similarity of MPC and RL will become evident from the discussion of Section 1.1. In particular, both fields are founded on DP, and both involve an on-line search algorithm and an off-line training process. Their differences are largely a matter of emphasis: RL has its roots on theories of game playing and learning, and places far more emphasis in off-line training, and discrete state and action spaces, while MPC has its roots in predictive on-line optimization and places far more emphasis in on-line play, and continuous state and control spaces. Other differences,

† "Model predictive control" is an abbreviation for the far more descriptive term "model-based predictive control." Here, the word "model" refers to either a mathematical model or a computer software model of a discrete-time dynamic system that evolves under the influence of control. The term "predictive" generally refers to taking into account the system's future, while applying control in the present.

‡ This view, while somewhat controversial within the AI community, has been supported by prominent figures outside the decision and control community, notably in public lectures by Yann LeCun, a pioneer in deep learning and co-recipient of the 2018 Turing Award. His statement is "Abandon RL in favor of MPC." Our related but somewhat different view, reflected in this book, is that "MPC is part of RL, and reversely, the most reliable part of RL is MPC."

which can also be traced to the respective origins of the two fields, are that RL includes a strong focus on approximation in policy space (which has been of peripheral interest in MPC), while MPC includes a strong focus on closed-loop system stability issues (which are hardly ever discussed in RL).

Supporting Literature

In this book, we will deemphasize mathematical proofs, and focus instead on visualizations and intuitive (but still rigorous) explanations. However, there is considerable related analysis, which supports our narrative, and can be found within a broad range of sources, which we describe next.

The author's approximate DP/RL books

[1] Bertsekas, D. P., 2019. Reinforcement Learning and Optimal Control, Athena Scientific, Belmont, MA.

[2] Bertsekas, D. P., 2020. Rollout, Policy Iteration, and Distributed Reinforcement Learning, Athena Scientific, Belmont, MA (can be downloaded from the author's website).

provide a far more detailed discussion of MPC, adaptive control, discrete optimization, and distributed computation topics, than the present book. Moreover, some other popular methods, such as temporal difference algorithms and Q-learning, are discussed in the books [1] and [2], but not in the present book.

The author's two-volume DP book

[3] Bertsekas, D. P., 2017. Dynamic Programming and Optimal Control, Vol. I, 4th Edition, Athena Scientific, Belmont, MA.

[4] Bertsekas, D. P., 2012. Dynamic Programming and Optimal Control, Vol. II, 4th Edition, Athena Scientific, Belmont, MA.

is an extensive source on the modeling and mathematical aspects of finite and infinite horizon DP. Modeling aspects and finite horizon problems are the principal focus of [3], while the mathematical aspects of infinite horizon problems are the principal focus of [4]. Both books [3] and [4] provide substantial accounts of approximate DP/RL methods. Thus these books are good entry points for a research-oriented reader that wishes to go into DP and its connections to RL more deeply.

Two of the author's recent research monographs are also highly relevant to our narrative:

[5] Bertsekas, D. P., 2022. Abstract Dynamic Programming, 3rd Ed., Athena Scientific, Belmont, MA (can be downloaded from the author's website).

This monograph focuses on the analytical aspects of abstract DP on which the Newton-based methodology is couched, and may serve as a mathe-

matical supplement to the present book. It also provides some supportive mathematical foundation for the more visually oriented monograph

[6] Bertsekas, D. P., 2022. Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control, Athena Scientific, Belmont, MA (can be downloaded from the author's website).

This monograph relies on intuitive visualization to convey the main ideas of our off-line training/on-line play/Newton's method conceptual framework for approximation in value space. It also focuses on model predictive and adaptive control, and associated issues of stability. Like our Section 1.5 of the present book, it is visually oriented, but goes in greater detail into various special cases, including pathological exceptions.

All of the above books are available as ebooks as well as in print form; see the Athena Scientific website. A lot of the material of the present book is adapted from these books. However, the books themselves collectively provide a far more detailed and mathematically rigorous presentation.

An extensive overview of the connections and applications of the conceptual framework of this book with model predictive and adaptive control is given in the paper

[7] Bertsekas, D. P., 2024. "Model Predictive Control, and Reinforcement Learning: A Unified Framework Based on Dynamic Programming," published in Proc. IFAC NMPC (can be downloaded from the author's website; a videolecture is available on-line).

Together with Chapter 1 of this book, it can be used as a starting point for a course in modern control system design, particularly a course that aims to touch upon the synergies of the subject with modern trends in artificial intelligence.

The present book can also be fruitfully supplemented by the extensive textbook and research monograph literature on RL. This literature is summarized in Section 1.8, and includes several accounts of RL that are based on alternative viewpoints of artificial intelligence, control theory, and operations research.

Structure of the Book - Course Adaptations

A key structural feature of this book is its modular organization, with a view towards flexibility, so it can be easily modified to accommodate changes in course content. In particular, the book is divided in two parts:

(1) A *foundational platform*, which consists of Chapter 1. It provides a selective overview of the approximate DP/RL landscape, and a starting point for a more detailed in-class development of other RL topics, whose choice can be at the instructor's discretion.

(2) An *in-depth coverage*, which consists of Chapters 2 and 3. It offers a deeper focus into specific methodologies. In particular, Chapter 2 de-

scribes primarily the deterministic and stochastic rollout techniques, as well as some related approximation in value methods. Chapter 3 addresses the use of neural networks and other approximation architectures for off-line training.

This modular structure allows for customization based on a course's focus. For example, instructors can use the foundational platform of Chapter 1 to build either more mathematically-oriented or less formal courses, depending on the needs of their class.

Let us also mention that the book contains more material than can be reasonably covered in class in one semester. This provides some flexibility to an instructor regarding the choice of material to present.

Videolectures, Slides, and Other Resources

The present book and my RL books above, were developed while teaching several versions of my course at ASU. Videolectures and slides from this course, as well as links to overview videolectures by the author are available from my website

http://web.mit.edu/dimitrib/www/RLbook.html

and provide a good supplement and companion resource to this book.

The website

https://space.bilibili.com/2036999141

contains much Chinese-language material, including research papers and videolectures. A translation of the author's 2019 "Reinforcement Learning and Optimal Control" book, translated into Chinese by Yuchao Li, has been published by Tsinghua Press.

An incisive review by Antonio Montano that focuses on the relations of the present book with alternative RL approaches and sources can be found at

https://4m4.it/collections/fkposts/bertsekas-a-course-in-reinforcement-learning/index.html

Thanks and Appreciation

The hospitable and stimulating environment at ASU contributed much to shaping my course during the period 2019-2025. I am thankful to several colleagues and students, and to my teaching assistants, Sushmita Bhattacharya, Sahil Badyal, and Jamison Weber, for their comments and suggestions.

I am especially thankful to Yuchao Li for extensive interactions, collaborative research, and proofreading support since 2019. Moreover, Yuchao has shared with me the lectures of the 2025 offering of the course, and has

undertaken to translate much of the course material into Chinese; see the website

https://space.bilibili.com/2036999141

Dimitri P. Bertsekas

Winter 2026

1

Exact and Approximate Dynamic Programming

Contents

1.1. AlphaZero, Off-Line Training, and On-Line Play . . . . p. 4
1.2. Deterministic Dynamic Programming . . . . . p. 10
1.2.1. Finite Horizon Problem Formulation . . . . . p. 10
1.2.2. The Dynamic Programming Algorithm . . . . . p. 14
1.2.3. Approximation in Value Space and Rollout . . . . p. 22
1.3. Stochastic Exact and Approximate Dynamic Programming p. 28
1.3.1. Finite Horizon Problems . . . . . p. 28
1.3.2. Approximation in Value Space for Stochastic DP . p. 34
1.3.3. Approximation in Policy Space . . . . . p. 38
1.3.4. Training of Cost Function and Policy . . . . . Approximations . . . . . p. 41
1.4. Infinite Horizon Problems - An Overview . . . . . p. 42
1.4.1. Infinite Horizon Methodology . . . . . p. 46
1.4.2. Approximation in Value Space - Infinite Horizon . p. 49
1.4.3. Understanding Approximation in Value Space . . . p. 55
1.5. Newton's Method - Linear Quadratic Problems . . . . . p. 56
1.5.1. Visualizing Approximation in Value Space - Region of . Stability . . . . . p. 62
1.5.2. Rollout and Policy Iteration . . . . . p. 70
1.5.3. Local and Global Error Bounds for Approximation in . Value Space . . . . . p. 74
1.5.4. How Approximation in Value Space Can Fail and . . . What to Do About It . . . . . p. 78

1.6. Examples, Reformulations, and Simplifications . . . . . p. 79
1.6.1. A Few Words About Modeling . . . . . p. 79
1.6.2. Problems with a Termination State . . . . . p. 83
1.6.3. General Discrete Optimization Problems . . . . . p. 85
1.6.4. General Finite to Infinite Horizon Reformulation . p. 89
1.6.5. State Augmentation, Time Delays, Forecasts, and . . . Uncontrollable State Components . . . . . p. 91
1.6.6. Partial State Information and Belief States . . . . p. 97
1.6.7. Multiagent Problems and Multiagent Rollout . . p. 102
1.6.8. Problems with Unknown or Changing Model - . . . . Adaptive Control . . . . . p. 107
1.6.9. Model Predictive Control . . . . . p. 117
1.7. Reinforcement Learning and Decision/Control . . . . p. 131
1.7.1. Differences in Terminology . . . . . p. 131
1.7.2. Differences in Notation . . . . . p. 133
1.7.3. Relations Between DP and RL . . . . . p. 134
1.7.4. Synergy Between Large Language Models and RL p. 135
1.7.5. Machine Learning and Optimization . . . . . p. 136
1.7.6. Mathematics in Machine Learning and Optimization p. 139
1.8. Notes, Sources, and Exercises . . . . . p. 140

This chapter has multiple purposes:

(a) *To provide an overview of the exact dynamic programming (DP) methodology, with a focus on suboptimal solution methods.* We will first discuss finite horizon problems, which involve a finite sequence of successive decisions, and are thus conceptually and analytically simpler. We will consider separately deterministic and stochastic finite horizon problems (Sections 1.2 and 1.3, respectively). The reason is that deterministic problems are simpler and have some favorable characteristics, which allow the application of a broader variety of methods. Significantly they include challenging discrete and combinatorial optimization problems, which can be fruitfully addressed with some of the reinforcement learning (RL) methods that are the main subject of the book. We will also discuss somewhat briefly the more intricate infinite horizon methodology (Section 1.4), and refer to the author's DP textbooks [Ber12], [Ber17a], the RL books [Ber19a], [Ber20a], and the neuro-dynamic programming monograph [BeT96] for a fuller presentation.

(b) *To summarize the principal RL methodologies, with primary emphasis on approximation in value space.* This is the approximate DP-based architecture that underlies the AlphaZero, AlphaGo, TD-Gammon and other related programs, as well as the Model Predictive Control (MPC) methodology, one of the principal control system design methods. We will also argue later (Chapter 2) that approximation in value space provides the entry point for the use of RL methods for solving discrete optimization and integer programming problems.

(c) *To explain the major principles of approximation in value space, and its division into the off-line training and the on-line play algorithms.* A key idea here is the connection of these two algorithms through the algorithmic methodology of Newton's method for solving the problem's Bellman equation. This viewpoint, recently developed in the author's "Rollout and Policy Iteration ..." book [Ber20a] and the visually oriented "Lessons from AlphaZero ..." monograph [Ber22a], underlies the entire course and is discussed for the simple, intuitive, and important class of linear quadratic problems in Section 1.5.

(d) *To overview the range of problem types where our RL methods apply, and to explain some of their major algorithmic ideas (Section 1.6).* Included here are partial state observation problems (POMDP), multiagent problems, and problems with unknown model parameters, which can be addressed with adaptive control methods.

This chapter will also discuss selectively some major algorithms for approximate DP and RL, including rollout and policy iteration. A broader discussion of DP/RL may be found in the RL books [Ber19a], [Ber20a], the DP textbooks [Ber12], [Ber17a], the neuro-dynamic programming mono-

graph [BeT96], alongside other textbooks referenced in Section 1.8.

The book reflects the author's decision/control and operations research orientation, which has in turn guided the choices of terminology, notation, and mathematical style throughout. On the other hand, RL methods have been developed within the artificial intelligence community, as well as the decision/control and operations research communities. Despite similarities in the mathematical structures of the problems that these communities address, there are notable differences in terminology, notation, and culture, which can be confusing to researchers entering the field. We have thus provided in Section 1.7 a glossary and an orientation to assist the reader in navigating the full range of the DP/RL literature.

1.1 ALPHAZERO, OFF-LINE TRAINING, AND ON-LINE PLAY

One of the most exciting recent success stories in RL is the development of the AlphaGo and AlphaZero programs by DeepMind Inc; see [SHM16], [SHS17], [SSS17]. AlphaZero plays Chess, Go, and other games, and is an improvement in terms of performance and generality over AlphaGo, which plays the game of Go only. Both programs play better than all competitor computer programs available in 2022, and much better than all humans. These programs are remarkable in several other ways. In particular, they have learned how to play without human instruction, just data generated by playing against themselves. Moreover, they learned how to play very quickly. In fact, AlphaZero learned how to play chess better than all humans and computer programs within hours (with the help of awesome parallel computation power, it must be said).

Perhaps the most impressive aspect of AlphaZero/chess is that its play is not just better, but it is also very different than human play in terms of long term strategic vision. Remarkably, AlphaZero has discovered new ways to play a game that has been studied intensively by humans for hundreds of years. Still, for all of its impressive success and brilliant implementation, AlphaZero is couched on well established theory and methodology, which is the subject of the present book, and is portable to far broader realms of engineering, economics, and other fields. This is the methodology of DP, policy iteration, limited lookahead, rollout, and approximation in value space.

It is also worth noting that the principles of the AlphaZero design have much in common with the work of Tesauro [Tes94], [Tes95], [TeG96] on computer backgammon. Tesauro's programs stimulated much interest in RL in the middle 1990s, and exhibit similarly different and better play than human backgammon players. A related impressive program for the (one-player) game of Tetris, also based on the method of policy iteration, is described by Scherrer et al. [SGG15], who relied on several earlier works, including those of Tsitsiklis and Van Roy [TsV96], and Bertsekas and Ioffe

[BeI96]. For a better understanding of the connections of AlphaZero and AlphaGo Zero with Tesauro's programs and the concepts developed here, the "Methods" section of the paper [SSS17] is recommended.

To understand the overall structure of AlphaZero and its connection to our DP/RL methodology, it is useful to divide its design into two parts: *off-line training*, which is an algorithm that learns how to evaluate chess positions, and how to steer itself towards good positions with a default/base chess player, and *on-line play*, which is an algorithm that generates good moves in real time against a human or computer opponent, using the training it went through off-line. We will next briefly describe these algorithms, and relate them to DP concepts and principles.

Off-Line Training and Policy Iteration

This is the part of the program that learns how to play through off-line self-training, and is illustrated in Fig. 1.1.1. The algorithm generates a sequence of *chess players* and *position evaluators*. A chess player assigns "probabilities" to all legal moves in a given position, representing the likelihood of each move being chosen. A position evaluator assigns a numerical score to a given position, predicting the player's chances of winning from that position. The chess player and the position evaluator are represented by two neural networks, a *policy network* and a *value network*, which accept a chess position and generate a set of move probabilities and a position evaluation, respectively.†

In more traditional DP terminology, a position is the state of the game, a position evaluator is a cost function that gives (an estimate of) the optimal cost-to-go at a given state, and the chess player is a randomized policy for selecting actions/controls at a given state.‡

The overall training algorithm is a form of *policy iteration*, a classical DP algorithm that will be of primary interest to us in this book. Starting from a given player, it repeatedly generates (approximately) improved players, and settles on a final player that is judged empirically to be "best"

---

† Here the neural networks play the role of *function approximators*; see Chapter 3. By viewing a player as a function that assigns move probabilities to a position, and a position evaluator as a function that assigns a numerical score to a position, the policy and value networks provide approximations to these functions based on training with data (training algorithms for neural networks and other approximation architectures are also discussed in the RL books [Ber19a], [Ber20a], and the neuro-dynamic programming book [BeT96]).

‡ One more complication is that chess and Go are two-player games, while most of our development will involve single-player optimization. However, DP theory extends to two-player games, although we will not focus on this extension. Alternately, we can consider training a game program to play against a known fixed opponent; this is a one-player setting, discussed further in Section 2.12.

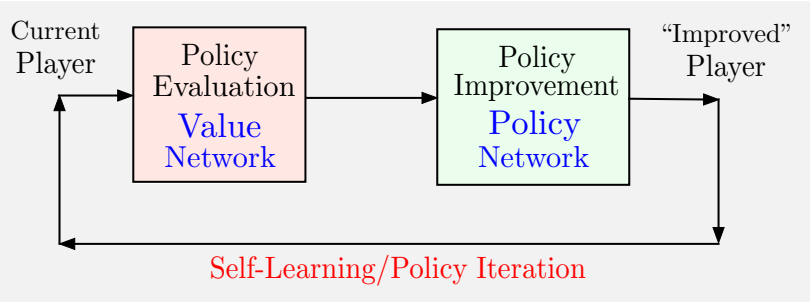


Figure 1.1.1 Illustration of the AlphaZero training algorithm. It generates a sequence of position evaluators and chess players. The position evaluator and the chess player are represented by two neural networks, a value network and a policy network, which accept a chess position and generate a position evaluation and a set of move probabilities, respectively.

out of all the players generated.† Policy iteration may be separated conceptually in two stages (see Fig. 1.1.1).

(a) *Policy evaluation*: Given the current player and a chess position, the outcome of a game played out from the position provides a single data point. Many data points are collected and used to train a value network, which then serves as the position evaluator for the player.

(b) *Policy improvement*: Given the current player and its position evaluator, trial move sequences are selected and evaluated for the rest of the game starting from many positions. An improved player is then generated by adjusting the move probabilities of the current player towards the trial moves that have yielded the best results. In AlphaZero this is done with a complicated algorithm called *Monte Carlo Tree Search*. However, policy improvement can also be done more simply. For example, all possible move sequences from a given position could be tried, extending forward a few moves, with the terminal

† Quoting from the paper [SSS17]: "The AlphaGo Zero selfplay algorithm can similarly be understood as an approximate policy iteration scheme in which MCTS is used for both policy improvement and policy evaluation. Policy improvement starts with a neural network policy, executes an MCTS based on that policy's recommendations, and then projects the (much stronger) search policy back into the function space of the neural network. Policy evaluation is applied to the (much stronger) search policy: the outcomes of selfplay games are also projected back into the function space of the neural network. These projection steps are achieved by training the neural network parameters to match the search probabilities and selfplay game outcome respectively." Note, however, that a two-person game player, trained through selfplay, may fail against a particular human or computer player that can exploit training vulnerabilities. This is a theoretical but rare possibility; see our discussion in Section 2.12.

position evaluated by the player's position evaluator. The move evaluations obtained in this way are used to nudge the move probabilities of the current player towards more successful moves, thereby obtaining data that is used to train a policy network that represents the new player.

Tesauro's TD-Gammon algorithm [Tes94] is similarly based on approximate policy iteration, but uses a different methodology for approximate policy evaluation, based on the TD(λ) algorithm. Unlike AlphaZero, TD-Gammon does not employ a policy network or MCTS. Instead, it relies solely on a value network, to replicate the functionality of a policy network by generating moves on-line via a one-step or two-step lookahead minimization. For a detailed description, see Section 8.6 of the neuro-dynamic programming book [BeT96].

On-Line Play and Approximation in Value Space - Rollout

Suppose that a "final" player has been obtained through the AlphaZero off-line training process just described. This player could, in principle, play chess against any human or computer opponent by generating move probabilities using its off-line-trained policy network. At any position, the player would simply select the move with the highest probability from the policy network. While this approach would allow the player to make decisions quickly, it would not be strong enough to defeat highly skilled human opponents. AlphaZero's extraordinary strength arises only when the off-line-trained player and position evaluator are embedded into another algorithm, which we refer to as the on-line player (see Fig. 1.1.2). At a given position, it generates a lookahead tree of all possible multiple move and countermove sequences, up to a given depth. It then evaluates the effect of the remaining moves by using the position evaluator of the off-line obtained value network.

The architecture of the final version of Tesauro's TD-Gammon program [TeG96] is similar to the one of AlphaZero, and uses an off-line neural network-trained terminal position evaluator; see Fig. 1.1.3. However, it also includes a middle portion, called "truncated rollout," which involves running a player for a few moves, before using the position evaluator. In effect, rollout can be viewed as an economical way to extend the length of the lookahead. In the published version of AlphaZero/chess [SHS17], there is also a rollout portion, but it is is rather rudimentary; the first portion (multistep lookahead) is quite long and efficiently implemented, so that a sophisticated rollout portion is not essential. Rollout plays a significant role in AlphaGo [SHM16], and is critically important in Tesauro's backgammon program, where long multistep lookahead is infeasible due to the rapid expansion of the lookahead tree.

Architectures that are similar to the ones of AlphaZero and TD-Gammon will be generically referred to as *approximation in value space*